



Ciência da Computação



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS

Universidade Federal de Goiás – UFG

Regional Catalão – RC

Departamento de Ciência da Computação

Disciplina: Compiladores – 6º período

Professor Sérgio

Luiz Eduardo Gonçalves Silva

luizeduardo.ufg@gmail.com

Matrícula: 201805035

Prova 01

0. Um compilador é um programa que traduz um programa de uma linguagem fonte para uma linguagem alvo. Já um interpretador é um programa que lê um programa fonte e produz como resultado a execução deste programa. A principal diferença entre os dois é que no caso do compilador, caso o programa contenha erros, a tradução não é feita e são reportados os erros. Já no caso do interpretador, uma mensagem de erro é exibida no momento que a linha com erro é interpretada.

1. O compilador recebe como entrada o programa fonte, em seguida esse programa passa pelo analisador léxico, o qual lê o fluxo de caracteres que compõem o programa fonte e os agrupa em sequências significativas (lexemas e tokens). Em seguida, é feita a análise sintática, onde é verificado se o programa está de acordo com a gramática da linguagem (GLC). Em seguida, é realizada a análise semântica, onde é validado se todos os identificadores estão declarados, se as operações envolvem tipos de dados compatíveis, entre outras coisas. Em seguida, o código passa pelo gerador de código intermediário, o qual gera um código intermediário objeto. Por fim, é gerado o código objeto alvo.

2. A análise léxica consiste em ler o fluxo de caracteres que compõem o programa fonte e os agrupar em sequências de lexemas e tokens. Por exemplo: adicionar variáveis à tabela de símbolos.

3. Porquê na análise sintática, é verificado se o programa está de acordo com a gramática da linguagem (gramática livre de contexto).

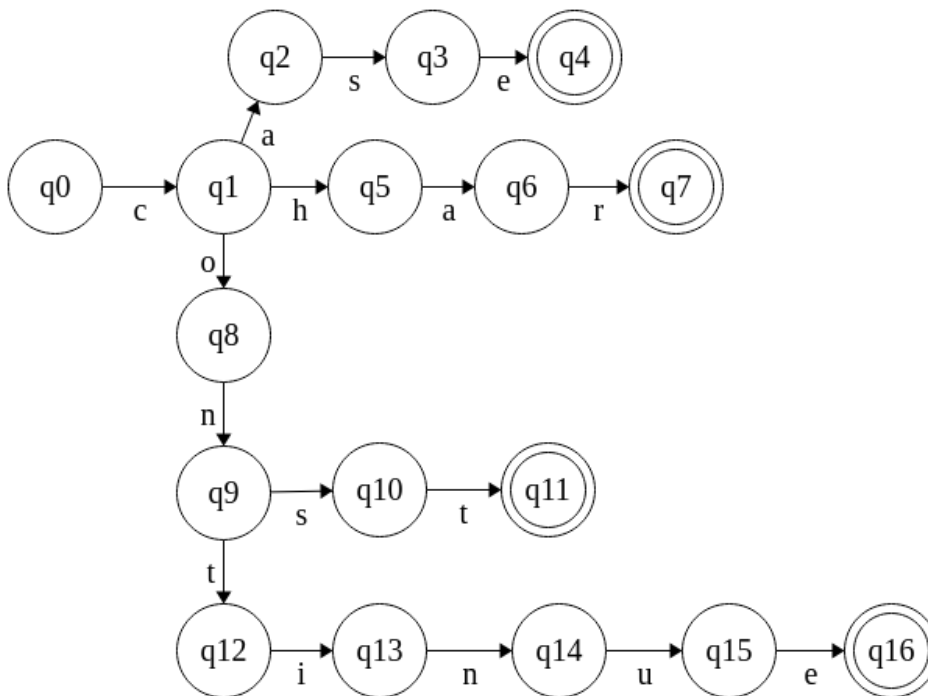
4. Quando o analisador léxico se depara com a palavra `for`, por exemplo, ao analisar letra por letra, e encontrar um `\b` (espaço em branco) em seguida da palavra, ele a classifica como palavra reservada (as quais são previamente conhecidas pelo analisador). Quando aos identificadores, como `cont`, por exemplo, o analisador pode avaliar se a palavra `cont` é seguida de algum operador lógico, por exemplo, como operador lógico de igualdade ou atribuição (`==` ou `=`).

5. Exemplos de erros que podem ser detectados pelo analisador léxico são: sequência de caracteres sem significado (não são identificadores, nem palavra reservada), os quais retornam um erro léxico.

6. As funções da análise sintática são: verificar se o programa está de acordo com a GLC e inserir informações de tipo das variáveis na tabela de símbolos.

7. Classes de tokens são padrões que pertencem à categoria das Linguagens Regulares, as quais são descritas por expressões regulares e reconhecidas por autômatos finitos. Portanto, expressões regulares servem para descrever linguagens regulares e autômatos finitos servem para reconhecer tais linguagens.

8.

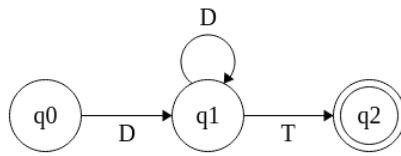


9. $D = [0-9]$

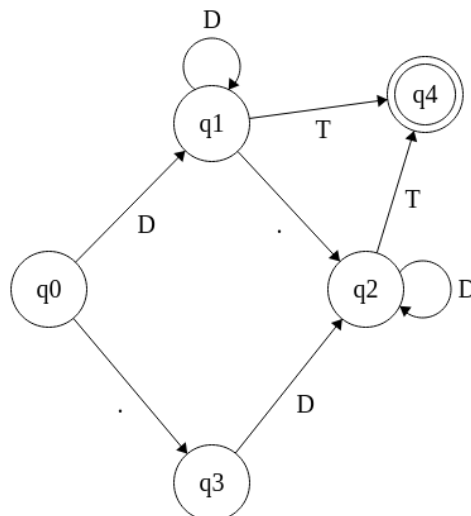
$L = [a-z, A-Z]$

$T = [\backslash b, \backslash t, \backslash n, +, -, *, /, >, <, =, |, :, ;, (,), \dots]$

- Números inteiros não sinalizados
 - Expressão Regular: (D^+)

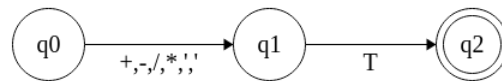


-
- Números reais não sinalizados (sem uso de potências de 10 ou notação científica)
 - Expressão regular: $(D^+ \mid D^+ \cdot \mid _D^+ \mid D^+ \cdot D^+)$



-
- Operadores aritméticos: +, -, /, *, (,)

- Expressão regular: (+ | - | / | * | " | “)



-
- Operador de atribuição: =
 - Expressão regular: (=)



-
- Identificadores
 - Expressão regular: (L | _) (L | _ | D)*



11. $(/*)((D|L)|"/"|(/| *)^*(*/))$, onde $D = [a-z, A-Z]$ e $L =$

13. ?

15. A linguagem gerada é: todas as strings com parênteses balanceados.

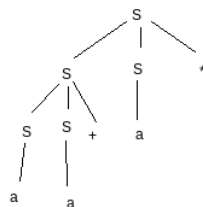


16.

E
 $E - T$
 $E - F$
 $E - (E)$
 $E - (E + T)$
 $E - (E + T * F)$
 $E - (E + T * 6)$
 $E - (E + F * 6)$
 $E - (E + 5 * 6)$
 $E - (T + 5 * 6)$
 $E - (F + 5 * 6)$
 $E - (4 + 5 * 6)$
 $T - (4 + 5 * 6)$
 $F - (4 + 5 * 6)$
 $3 - (4 + 5 * 6)$

17.

1) $S = lm \Rightarrow SS^* \Rightarrow SS + S^* \Rightarrow aS + S^* \Rightarrow aa + S^* \Rightarrow aa + a^*$
 2) $S = rm \Rightarrow SS^* \Rightarrow Sa^* \Rightarrow SS + a^* \Rightarrow Sa + a^* \Rightarrow aa + a^*$

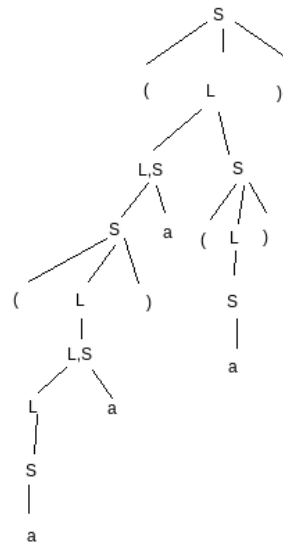


3)

4) Não, pois somente uma árvore de derivação foi encontrada para a linguagem.

5) O conjunto de todas as expressões pós-fixadas consiste em adição e multiplicação.

18.5.

$$1) S = \text{Im} \Rightarrow (L) \Rightarrow (L, S) \Rightarrow (L, S, S) \Rightarrow ((S), S, S) \Rightarrow ((L), S, S) \Rightarrow ((L, S), S, S) \Rightarrow ((S, S), S, S) \Rightarrow ((a, S), S, S) \Rightarrow ((a, a), S, S) \Rightarrow ((a, a), a, S) \Rightarrow ((a, a), a, (L)) \Rightarrow ((a, a), a, (S)) \Rightarrow ((a, a), a, (a))$$
$$2) S = rm \Rightarrow (L) \Rightarrow (L, S) \Rightarrow (L, (L)) \Rightarrow (L, (a)) \Rightarrow (L, S, (a)) \Rightarrow (L, a, (a)) \Rightarrow (S, a, (a)) \Rightarrow ((L), a, (a)) \Rightarrow ((L, S), a, (a)) \Rightarrow ((S, S), a, (a)) \Rightarrow ((S, a), a, (a)) \Rightarrow ((a, a), a, (a))$$


3)

4) A gramática não é ambígua, pois só foi possível encontrar uma árvore de derivação para a linguagem.

5) Equivalente a uma tupla em Python.

18.7. Expressões booleanas não ambíguas. A linguagem descreve o conjunto de todas as cadeias de 0s e é da forma xy , onde x^ny e x e y são do mesmo comprimento.

19. ?

20. Não existe fator comum esquerdo.

21.4. ?

21.5. Não existe fator comum esquerdo.